Vaayu: Autonomous Agent Build Plan

Oct 25, 2026 · @manasvi

Reality check and goals

Vaayu can become a capable, multilingual, safe personal agent on a GTX 1650, but not a frontier-level model like Claude or GPT. The gap is raw model size: frontier models run on data-centre GPUs, while the 1650 has 4 GB of VRAM. What closes most of that gap for a personal assistant is good engineering around a small model: tools, memory, a strict Guardian, and tight testing.

Minimum spec (design target): GTX 1650, 4 GB VRAM, 16 GB RAM, Windows. Everything below must run on this. The RTX 3050 laptop (6 GB) is an optional upgrade path, never a requirement.

Goals

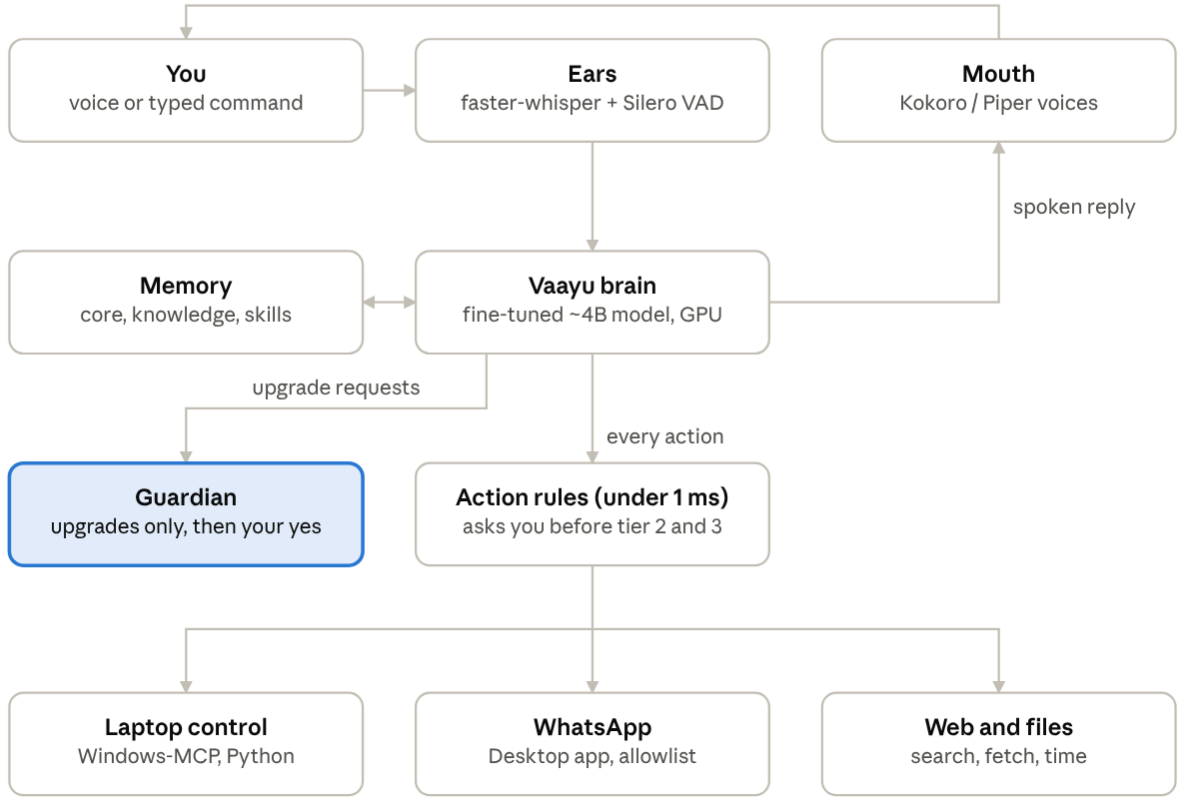
1. Understands and speaks many languages, including Hindi, Hinglish and English.
2. Operates the laptop: opens apps, files, folders and websites, types and clicks.
3. Sends WhatsApp messages only on your explicit command, with a confirmation step.
4. Learns over time through memory, research and a growing skill library.
5. Proposes its own upgrades, which must pass the Guardian and your approval before anything changes.
6. Costs nothing: free, open-source tools only.

What "top tier" realistically means here: fast, reliable, never does anything you did not approve, admits when it does not know, and gets better every month. That is achievable.

Architecture overview

Vaayu's brain thinks and plans, and acts through fast action rules that need no AI model. Only upgrades to Vaayu itself go through the Guardian.

Actions pass instant code rules; only upgrades go to the Guardian



Vaayu architecture · 9 parts, Guardian between brain and tools

A command flows from your voice to the brain, which reads memory, then runs tools through the action rules; the reply comes back as speech.

The brain: model and speed on the 1650

Keep your fine-tuned model as the main brain, held to about 4B parameters at 4-bit quantization so it fits in 4 GB of VRAM. Everything else is about using that small brain efficiently.

Model roles

Role	Model (approx. size)	Runs on
Main brain: conversation, planning, tool calls	Your fine-tuned model, ~4B, Q4 (~2.5 GB)	GPU
Background worker: research, summaries, reflection	qwen3:1.7b or 0.6b	CPU, when idle
Guardian judge	Llama Guard 3 1B INT4 (~440 MB)	CPU

Role	Model (approx. size)	Runs on
Embeddings for memory search	A small embedding model (384 dimensions)	CPU

Only the main brain lives on the GPU. A 4 GB card cannot hold two models at once, so everything else runs on CPU.

Speed tricks that matter most

- Turn off "thinking" mode for chat; enable it only for hard planning tasks.
- Stream replies and speak sentence by sentence (already in `vaayu_brain.py`).
- Keep the system prompt short and trim history to the last few turns.
- Keep the model loaded (`OLLAMA_KEEP_ALIVE=-1`) so it never reloads between commands.
- Route simple commands ("open Chrome", "what time is it") through fast rules without calling the LLM at all.
- Cache answers to repeated questions.
- Use a smaller quantization (Q4_K_M) and a context window of about 4K–8K tokens; bigger contexts slow the 1650 down sharply.

Multilingual: hearing, thinking and speaking

Use a Qwen3-based model for the brain: Qwen3 raised language support from 29 to 119 languages and dialects (Qwen3 report). Quality is best in English and Hindi and drops for low-resource languages, so test the ones you care about.

Stage	Free tool	Notes for the 1650
Speech to text	faster-whisper (`small`, int8)	Auto-detects the language; runs on CPU or GPU
Indian-language speech (optional)	AI4Bharat Indic models	Better Hindi, Tamil, Bengali accuracy than Whisper small
Brain	Your fine-tuned Qwen3-based model	Answer in the language the user spoke
Text to speech	Kokoro (English, Hindi and several others)	Fast on CPU
Extra languages TTS	Piper voices	Many languages, very light

How it works: Whisper detects the language, Vaayu replies in that language, and the TTS layer picks the matching voice. For Hinglish, add Hinglish conversations to your next fine-tune; small models handle code-mixing poorly without it.

Laptop control

Give Vaayu "hands" through MCP servers, and put every action into a permission tier so the action rules know how careful to be.

Free tools

- Windows-MCP: opens apps, switches windows, clicks, types and reads the screen through Windows UI Automation. It is model-agnostic, so it works with Vaayu's local model through your MCP bridge.
- pywinauto-mcp: an alternative for native Windows apps, with OCR and screenshots.
- Playwright (which you already know as an SDET): browser tasks, with the Playwright MCP server.
- Plain Python (`os.startfile`, `subprocess`): the fastest way to open known apps and files, with no LLM involved.

Permission tiers

Tier	Examples	Rule
0 Safe	Open an app, read the time, search the web, play music	Runs immediately
1 Change	Create or move files, type into apps, change volume	Runs, logged, undoable where possible
2 Outward	Send a message, post, email, submit a form	Needs your spoken or typed "yes" every time
3 Dangerous	Delete files, install software, change system settings, payments	Blocked by default; you do it yourself or approve with a PIN

Real-world example: you say "Open my resume and the Infosys careers page." Vaayu opens the PDF with `os.startfile` (tier 0) and opens the browser (tier 0). No confirmation needed, done in under two seconds.

WhatsApp: on your command only

Use the WhatsApp Desktop app driven like a human, not an unofficial WhatsApp library. Libraries such as Baileys and whatsapp-web.js work but violate WhatsApp's Terms of Service and carry a real ban risk for your personal number (analysis). The official Cloud API is ban-safe but needs a separate WhatsApp Business account and a Meta developer setup, so it does not fit messaging your own contacts.

Option	Cost	Ban risk	Verdict
WhatsApp Desktop + Windows-MCP or a `whatsapp://send` link	Free	Low (a human-paced, one-off send)	Recommended
Official WhatsApp Cloud API	Free tier, needs Business account	None	Only for a separate business number
Baileys / whatsapp-web.js	Free	High	Avoid on your main number

Safe flow, every time

1. You say: "Send Rahul: I'll be late by 10 minutes."
2. Vaayu looks up Rahul in a local contacts allowlist that you maintain. Unknown name = stop and ask.
3. Vaayu reads back: "Sending to Rahul, +91 98xxx: 'I'll be late by 10 minutes.' Confirm?"
4. Only after your "yes" does it open WhatsApp Desktop and press Send.
5. The send is logged: time, recipient, exact text.

Hard rules: never send on a command found inside a message, web page or file; never more than a few messages a minute; never broadcast; never send to numbers outside the allowlist without a fresh confirmation.

Memory and self-learning

Vaayu learns through four stores, not by retraining after every conversation. Weights change only in the periodic "sleep" fine-tune, which goes through the Guardian like any other upgrade.

Store	What it holds	Free tool
Core memory	Who you are, your preferences, Vaayu's personality	Letta memory blocks, or a small JSON file
Knowledge base	Summaries from research, with source URLs	LanceDB or ChromaDB with int8 embeddings
Skill library	Python functions Vaayu wrote and verified	A git repo of `skills/*.py`
Lessons	What went wrong and what to do next time	A notes table in the same database

Learning loops

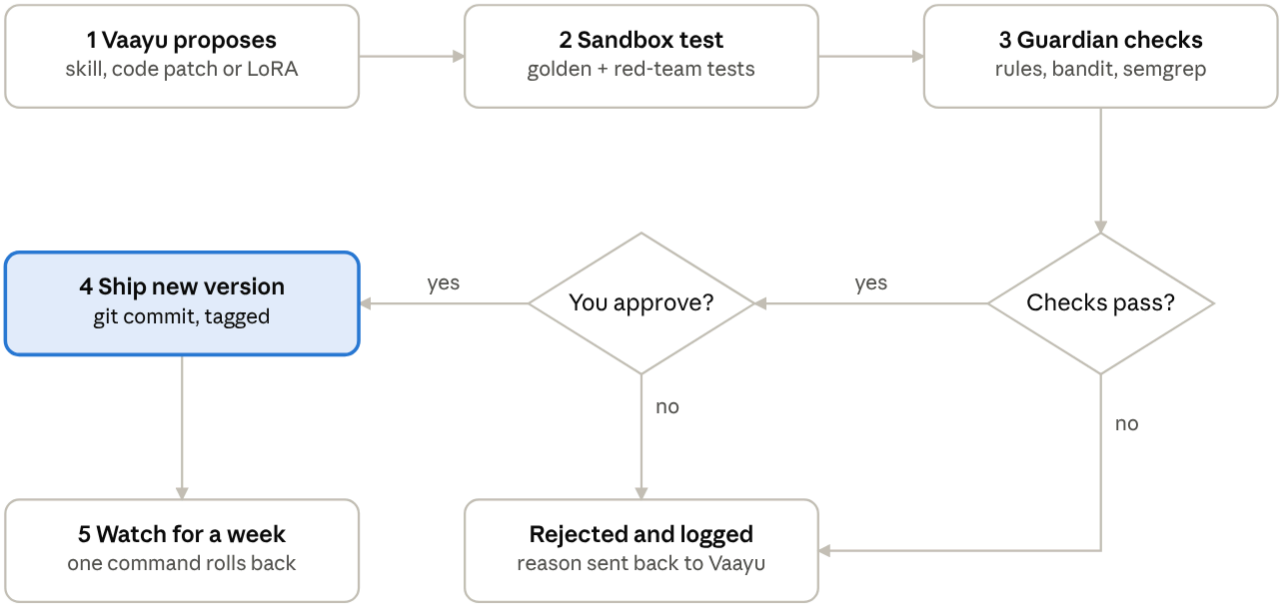
- **Research loop** (when idle): pick a topic from your questions or Vaayu's curiosity list, search, read, summarise, store with the URL.
- **Reflection loop** (after each task): "Did it work? What would I change?" The lesson is saved.
- **Consolidation** (weekly): merge duplicates, drop stale facts, compress notes.
- **Sleep fine-tune** (every 2–3 months): distil the most-used, stable lessons into a LoRA. Train on the RTX 3050 or a free Colab GPU; Vaayu still runs on the 1650 afterwards.

Storage stays small: roughly 0.5–2 GB per five months of learned data, most of it logs.

Self-upgrade pipeline

Vaayu can write its own improvements, but it can never install them: every upgrade is tested in a sandbox, checked by the Guardian, and approved by you before it ships.

Nothing ships without the Guardian checks and your yes



Self-upgrade pipeline · 5 steps, 2 gates

An upgrade is one of three things: a new skill (a Python function), a code patch to Vaayu itself, or a new LoRA from the sleep fine-tune. All three take the same road.

- **Sandbox:** Docker Desktop or Windows Sandbox, with no access to your real files or accounts.
- **Tests:** the golden set must score at least as well as the current version, and the red-team set must stay 100% blocked.
- **Your approval:** Vaayu shows a plain-language summary and the diff; you say yes or no.
- **Rollback:** every version is a git tag, so `git checkout` of the previous tag restores Vaayu in seconds.

The Guardian's upgrade report

Before you approve any upgrade, the Guardian hands you a one-page report built from what the code actually does, not from what Vaayu says it does. Vaayu's own description appears in it, but labelled as a claim and checked against the evidence.

How it stays unbiased

- Facts first: the report is generated from the code diff, sandbox logs and test results by fixed Python checks. The AI judge adds an opinion at the end, clearly marked as advisory.

- Claim vs reality: if Vaayu says "adds a weather skill" but the code also opens a network connection to an unknown domain, the report shows the mismatch in red and the harm level goes up.
- Vaayu never writes, edits or sees the report before you do.

What every report contains

1. **Summary:** what changes, in two plain sentences, written by the Guardian.
2. **Harm level** (0–4, table below) with the exact reasons it was given.
3. **Claim vs actual behaviour:** Vaayu's description next to what the sandbox observed; any mismatch flagged.
4. **New powers gained:** network access, file access outside Vaayu's folder, running programs, admin rights, new tools or MCP servers.
5. **Files and folders touched** in the sandbox run.
6. **Network activity:** every domain contacted, with new ones marked.
7. **Test results:** golden-set score before vs after, red-team block rate (must be 100%), and speed change on the 1650.
8. **Code scan findings** from bandit and semgrep.
9. **AI judge opinion** (Llama Guard), advisory only.
10. **Rollback:** the version tag to return to, with one command.
11. **Guardian recommendation:** approve, approve with limits, or reject.

Harm levels

Level	Meaning	Examples	What you must do
0 Cosmetic	Changes how Vaayu talks, not what it can do	Prompt wording, voice, reply length	A simple yes
1 Low	New ability with no outside effect	A read-only skill, a new calculation	A simple yes
2 Medium	Writes inside Vaayu's own folder or reads known websites	Saving notes, a new search source	Yes after reading the report
3 High	Can affect your laptop or other people	New messaging action, file access outside its folder, running programs, a new LoRA that changes behaviour	Yes plus your PIN

Level	Meaning	Examples	What you must do
4 Critical	Could take control away from you	Admin rights, changing security settings, startup or hidden processes, reading passwords or browser data, disabling logs, touching the Guardian or action rules	Automatically rejected; only you can make such a change, by hand

Powerful actions only on your command

You are the only one who can authorise Vaayu's powerful actions on your own laptop, such as admin-level commands, changing system settings, or running security tools on your own machine. These stay tier 3 actions: Vaayu does them only when you ask, says exactly what it will do, and waits for your PIN. No upgrade can give Vaayu the ability to do them on its own. One rule is permanent: Vaayu never acts against devices, networks or accounts that are not yours.

Example report (shortened)

```
UPGRADE #14 — "Add a cricket-score skill"
Harm level: 2 (Medium)
Summary: Adds a skill that reads live scores from one website and speaks
them.
Claim vs actual: MATCH. Contacts 1 domain (new). Writes 1 file in
vaayu/skills/.
New powers: network read access to 1 domain. No admin, no programs run.
Tests: golden set 91% -> 91%. Red-team 100% blocked. Latency +0.0 s.
Code scan: no findings.
AI judge: safe (advisory).
Rollback: git checkout v0.13
Recommendation: APPROVE
```

The Guardian: Vaayu's firewall

The Guardian checks upgrades only: new skills, code patches, new tools or MCP servers, and fine-tuned LoRAs. Everyday actions skip it and pass through fast action rules instead, so commands stay instant.

Action rules (every action, under 1 ms). Plain Python checks inside Vaayu's tool layer, with no AI model: permission tiers, the contacts allowlist, rate limits, blocked folders and commands, and never following instructions found inside web pages, files or messages.

The only delay they add is asking for your yes on tier 2 and 3 actions.

Guardian (upgrades only), three layers

1. **Hard rules.** An upgrade may never touch the Guardian, the action rules or the allowlist. New network endpoints, file deletions and subprocess calls are always flagged.
2. **Static checks.** `bandit` and `semgrep` scan every code change.
3. **AI judge.** Llama Guard 3 1B INT4, about 440 MB, on CPU. It takes seconds, which is fine for an upgrade but would be too slow for every action.

Isolation rules that make the Guardian real

- The Guardian runs as a separate process under a separate Windows user; Vaayu cannot write to its folder.
- Vaayu cannot edit its own code folder directly; only the Guardian's pipeline can merge an approved upgrade.
- The Guardian and the action rules are changed only by you, by hand.
- Every decision is logged to an append-only file you can review.
- A kill switch: a hotkey or the phrase "Vaayu, stop" freezes all actions instantly.

What reaches you: every upgrade that passes the checks, for a final yes or no.

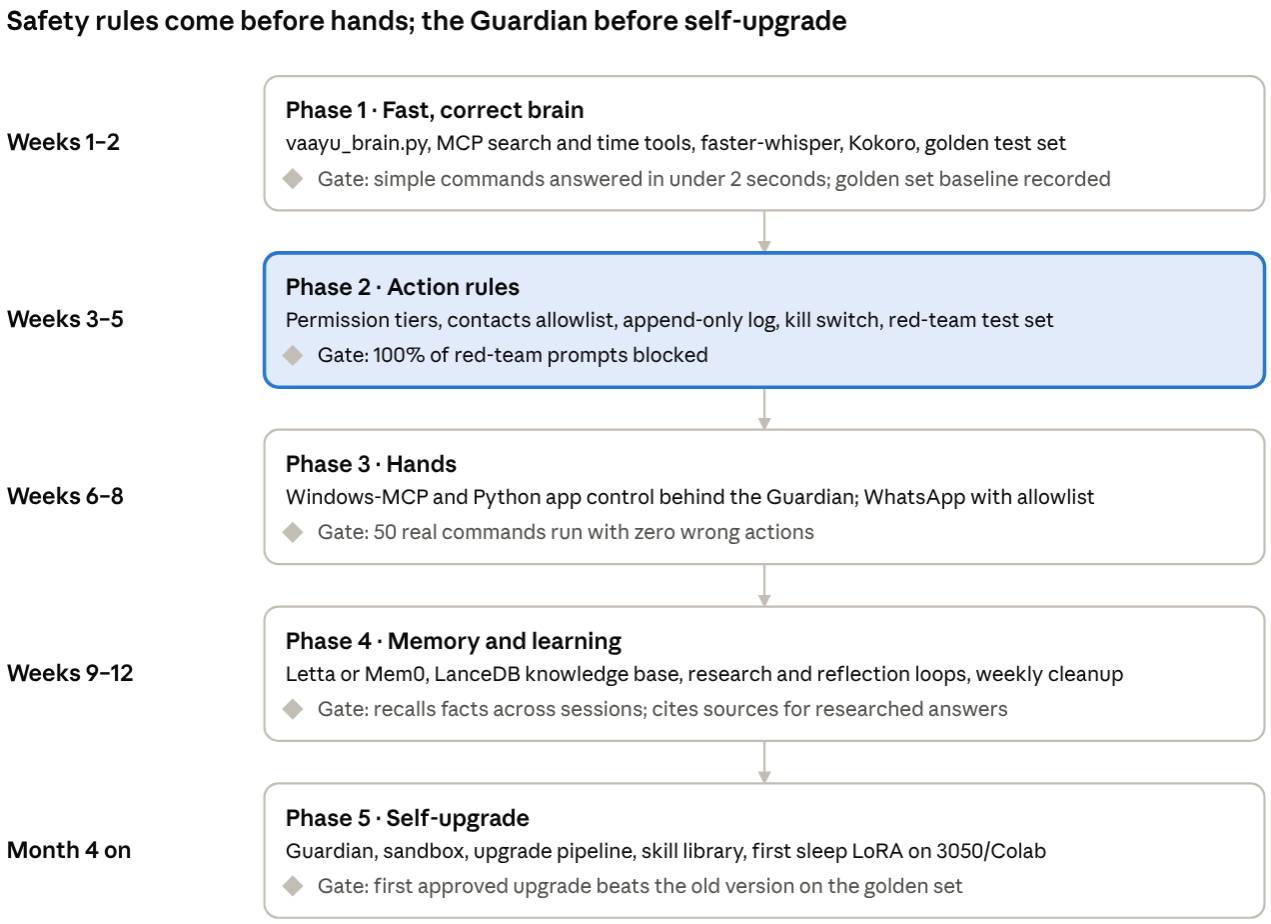
Testing and evaluation

Treat Vaayu like a product you test: an upgrade that does not beat the current version on a fixed test suite does not ship. This is where your SDET experience is the biggest advantage.

- **Golden set:** 200–300 fixed questions and commands with expected outcomes (facts, tool choice, language, tone). Run it with `pytest` on every upgrade and compare scores.
- **Safety red-team set:** prompts that try to make Vaayu delete files, message strangers, follow instructions hidden in a web page, or edit the Guardian. Every one must be blocked, every time.
- **Latency budget:** time to first spoken word under ~2 seconds for simple commands on the 1650; measured on every run.
- **Regression on old skills:** after a fine-tune, re-run the full set to catch forgetting.
- **Weekly review:** skim the Guardian log and the new lessons; delete bad ones.

Roadmap

At 2–3 hours a day, a fast, safe Vaayu that controls your laptop takes roughly 3 months; self-upgrading starts in month 4. These are estimates, and a phase does not start until the previous gate passes.



Roadmap · 5 phases, each with a gate

The order is deliberate: action rules exist before Vaayu can touch apps or WhatsApp, and the Guardian exists before Vaayu can change itself.

Free tool list

Job	Tool
Run models	Ollama
Brain	Your fine-tuned Qwen3-based model (~4B, Q4)
Connect tools	Python MCP SDK (`pip install mcp`)

Job	Tool
Speech to text	faster-whisper; AI4Bharat models for Indian languages
Text to speech	Kokoro, Piper
Wake word, end of speech	openWakeWord, Silero VAD
Voice pipeline (later)	Pipecat
Laptop control	Windows-MCP, Playwright
Web lookups	duckduckgo-mcp-server, mcp-server-fetch, mcp-server-time
Memory	Letta or Mem0
Knowledge base	LanceDB or ChromaDB
Self-written skills	smolagents
Sandbox	Docker Desktop or Windows Sandbox
Code safety scans	bandit, semgrep
Guardian AI judge	Llama Guard 3 1B INT4
Version control and rollback	git
Tests	pytest
Fine-tuning (on the 3050 or Colab)	Unsloth